

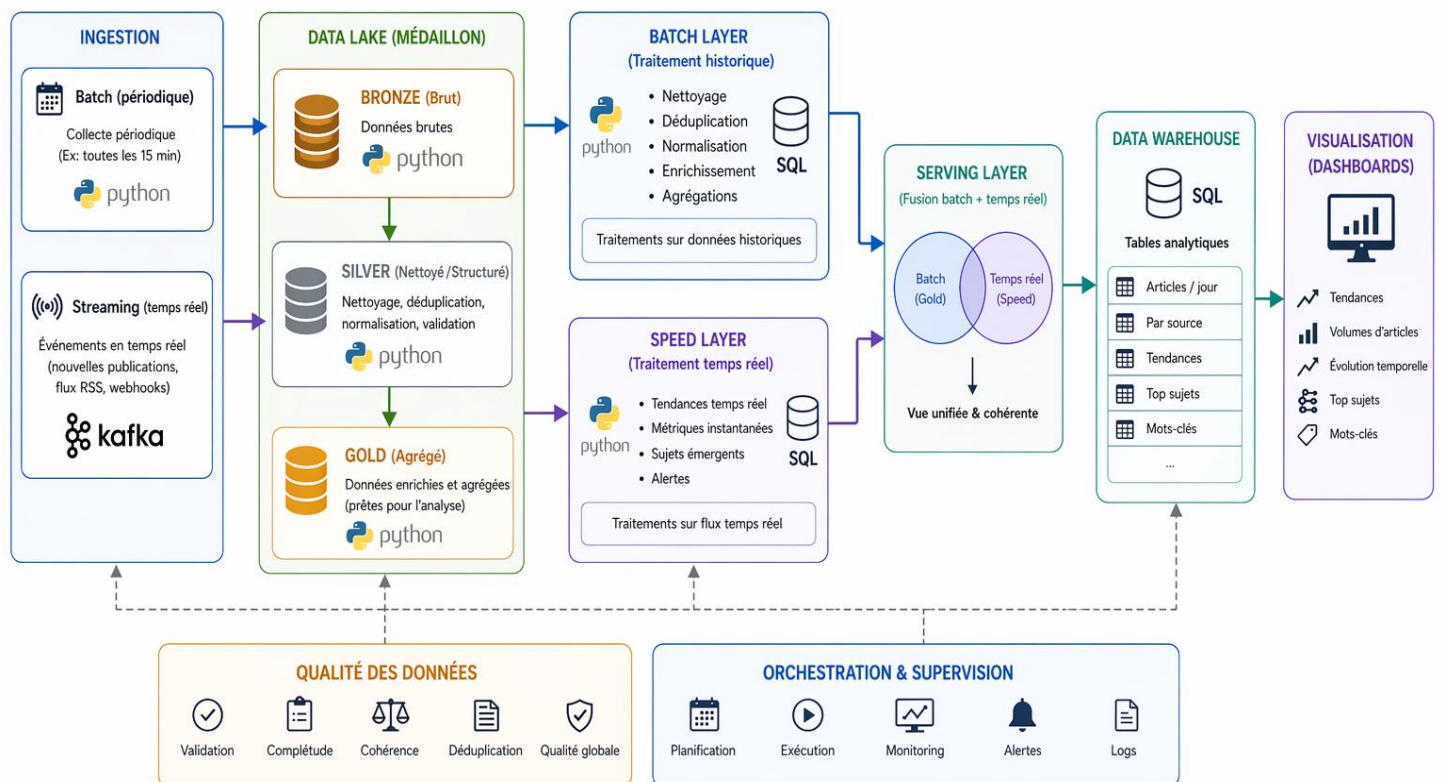
Projet : Architecture de données

Les médias publient chaque jour des milliers d'articles. Ces données peuvent être exploitées pour :

- identifier les tendances d'actualité
- analyser les thèmes dominants
- suivre les événements en temps réel
- détecter les fake news

Le projet consiste à concevoir une plateforme Big Data capable de collecter automatiquement des articles de presse à partir de plusieurs sites d'actualité, puis de stocker, transformer et analyser ces données afin d'identifier les tendances médiatiques.

La solution doit être basée sur une architecture distribuée moderne intégrant ingestion, stockage, transformation, qualité de données, gouvernance et visualisation.



Les tâches :

- Collecter des données web (**web scraping**)
- Implémenter une **architecture de données distribuée**
- Mettre en place un **Data Lake**
- Construire une **architecture Médaille**
- Implémenter **ETL / ELT**
- Gérer **batch et streaming**
- Assurer **qualité et gouvernance des données**

Exemples de sites d'actualité

- Marocain : Akhbarona, Hespress, Lakom, Barlamane

- International : Al Jazeera, CNN, BBC News, Reuters

Données collectées :

- Titre article
- Auteur
- Date publication
- Catégorie
- Contenu
- Source
- Url
- ... etc

1. Data Sources

Un scraper doit être développé pour collecter automatiquement les articles.

Scraping web : Python + BeautifulSoup / Scrapy

2. Ingestion

Les données doivent être ingérées via deux modes :

- **Batch ingestion** : Scraping programmé toutes les heures.
- **Streaming ingestion** : Chaque article publié est envoyé comme **événement**.

3. Data Lake

Les données brutes doivent être stockées dans un Data Lake afin de conserver l'historique complet des informations collectées.

Stockage possible :

- MinIO
- HDFS
- S3
- ...etc

4. Architecture Médaille

- **Bronze** : Articles bruts.

- **Silver** : Nettoyage

- Suppression HTML
- Normalisation texte
- Détection langue

- **Gold** : Tables analytiques

- Tendances news
- Top sujets

- Nombre articles par source
- ...etc

5. Transformation des données

Les transformations doivent être réalisées avec python pour nettoyer, normaliser et enrichir les données.

6. Orchestration

Les pipelines doivent être orchestrés avec Apache Airflow ou Nifi afin de planifier et superviser les traitements.

7. Data Warehouse

Les données analytiques doivent être stockées dans un Data Warehouse pour faciliter les analyses décisionnelles.

Tables analytiques :

- Articles par jour
- Articles par thème
- Articles par pays

8. Visualisation

Des tableaux de bord doivent être créés afin de visualiser les tendances et indicateurs clés issus des articles collectés.

Dashboards :

- Tendances d'actualité
- Nombre articles par source
- Mots clés les plus fréquents

9. Qualité des données

Des contrôles doivent être mis en place pour vérifier la complétude, la cohérence et la validité des données.

- Tests :

- Article sans titre
- Date manquante
- Contenu trop court

- Dimensions :

- Complétude
- Cohérence
- Validité

10. Gouvernance

Les données doivent être documentées et leur traçabilité assurée afin de garantir la transparence du pipeline.

11. Déploiement

La plateforme doit être déployée dans un environnement **conteneurisé** basé sur Docker. Chaque composant de l'architecture (scrapers, broker de messages, Data Lake, moteur de traitement, orchestrateur, base de données analytique, outil de visualisation) est packagé sous forme d'image Docker dédiée, puis orchestré via un fichier **docker-compose.yml** unique. Ce fichier définit les dépendances entre services, les volumes persistants, les réseaux isolés et les variables d'environnement, garantissant ainsi un déploiement reproductible et portable entre les différents environnements.

Déploiement avec Kubernetes (optionel)

Pour une mise à l'échelle en production, la plateforme peut être déployée sur un cluster **Kubernetes** à l'aide de **Helm Charts**. Cette approche permet de gérer automatiquement la scalabilité des composants, la haute disponibilité, la persistance des données via des volumes, ainsi que la sécurité grâce aux Secrets et ConfigMaps. Chaque couche fonctionnelle est isolée dans un namespace dédié, facilitant la maintenance et la supervision.

11.3 Monitoring (optionel)

Une stack de monitoring basée sur **Prometheus et Grafana** doit être intégrée à la plateforme. Prometheus collecte en continu les métriques des différents composants (ingestion, traitement, stockage, orchestration), tandis que **Grafana** fournit des tableaux de bord visuels permettant de superviser en temps réel la santé des pipelines, les performances des services et l'utilisation des ressources.

12. Livrables attendus

À l'issue du projet, les livrables suivants doivent être remis :

- **Présentation détaillée du projet** incluant le contexte, l'architecture et les choix technologiques
- **Schéma d'architecture** illustrant les couches et les flux de données
- **Code source** versionné sur Git
- **Fichiers de déploiement** (Docker, Kubernetes)
- **Documentation technique** d'installation et d'utilisation
- **Dashboards de visualisation** des indicateurs clés
- **Démonstration fonctionnelle** du pipeline de bout en bout